

TSFS12 Hand-in 1: Discrete Path Planning in a Structured Road Network

MAXENCE MAURY

E-mail: maxma741@student.liu.se

LiU-ID: maxma741

JESSY SANFILIPPO

E-mail: jessa070@student.liu.se

LiU-ID: jessa070

September 14, 2025

Contents

1	BASIC VERSION	1
1.1	Exercise 3.1	1
1.2	Exercise 3.2	1
1.3	Exercise 3.3	2
1.4	Exercise 3.4	3
1.5	Exercise 3.5	3
1.6	Exercise 3.6	4
2	APPENDIX	5

1 BASIC VERSION

1.1 Exercise 3.1

A discrete planning problem is defined by a state space X , an action space $U(x)$ for each state $x \in X$, and a transition function $f(x, u)$ that leads to a new state x' .

In this route planning problem:

- X is the set of intersections or road nodes, each identified by its coordinates (latitude, longitude).
- $U(x)$ is the set of roads (edges) leaving a given intersection.
- $f(x, u)$ represents moving from one intersection to a connected neighbor via a road.

The provided `f_next` function directly corresponds to $f(x, u)$: for a given node x , it returns the possible next nodes x' , the action u , and the distance d for each transition.

1.2 Exercise 3.2

We implemented the required planners: Breadth First, Dijkstra, Best First, A*, and also compared with Depth First.

Mission complexity. Easy missions are typically characterized by a short distance between start and goal and few alternative routes. Complex missions involve longer paths, multiple possible detours, and dense parts of the road network.

Results. We collected plan length, number of expanded nodes, and planning time for each algorithm. We also added in the appendix a test of each algorithm to compare them.

Plots.

- **Plan length vs planning time:** see Figure 1.
- **Planning time vs expanded nodes:** see Figure 2.

Algorithm	Plan length (m)	Expanded nodes	Planning time (ms)
Breadth First	5754.1	9101	1329.2
Dijkstra	5085.6	9847	1451.0
Best First	5386.1	283	82.5
A*	5085.6	3251	574.0
Depth First	24240.1	2180	347.7

Table 1: Comparison of planners on sample missions.

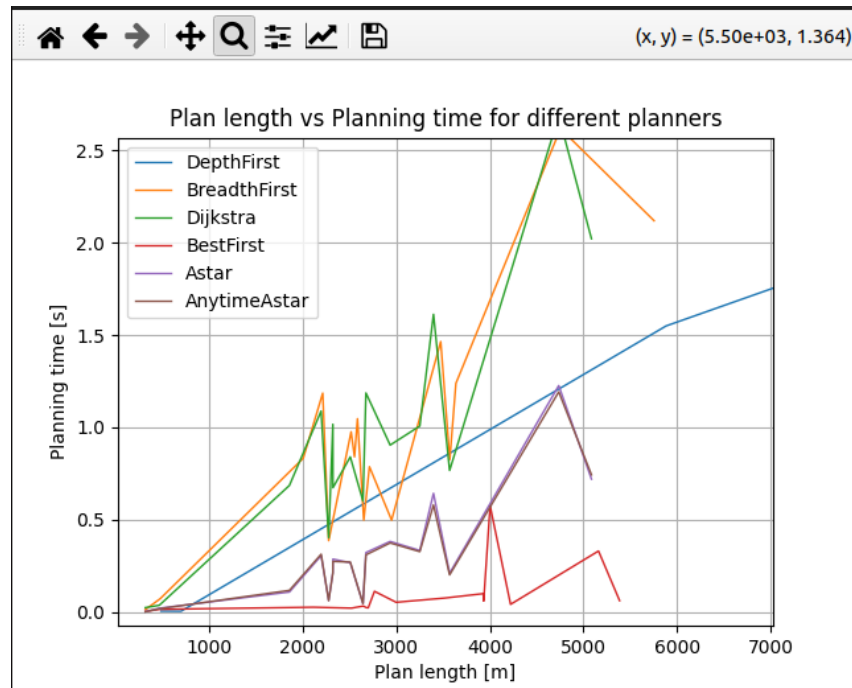


Figure 1: Plan length vs planning time for all planners.

Reflection. Dijkstra guarantees optimality but expands a huge number of nodes, leading to long runtimes. Best First is fast but can yield suboptimal paths. A* balances both by using a heuristic; it still guarantees optimality but visits fewer nodes. Depth First is unsuitable, as it produces very long and impractical paths.

1.3 Exercise 3.3

Properties.

- **Breadth First:** explores evenly but is expensive in large maps.
- **Dijkstra:** optimal but very costly in time and nodes.
- **Best First:** very fast but can be far from optimal.
- **A*:** efficient compromise, optimal with admissible heuristic.
- **Depth First:** impractical, produces excessively long detours.

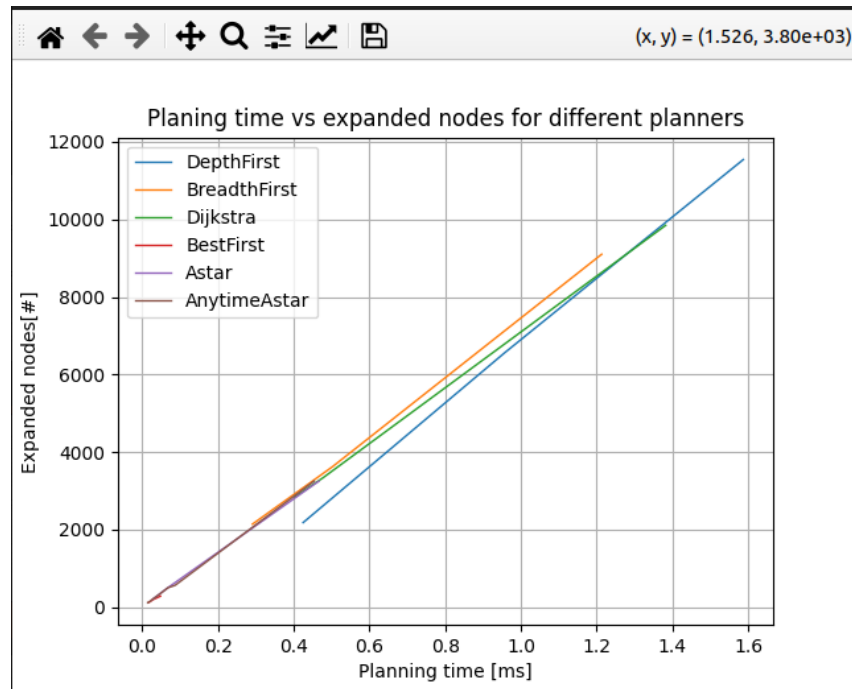


Figure 2: Planning time vs expanded nodes for all planners.

1.4 Exercise 3.4

Admissibility. If the heuristic $h(x)$ is not admissible ($h(x) > h^*(x)$), A* may overlook the optimal path and return a suboptimal solution.

Consistency. If the heuristic is not consistent, the algorithm may need to re-expand nodes multiple times, which greatly reduces efficiency.

1.5 Exercise 3.5

For labyrinth-like environments, Euclidean distance underestimates the real cost because obstacles (walls, dead-ends) make the true path much longer. For example, two nodes may be geographically close, but the only valid path requires a large detour.

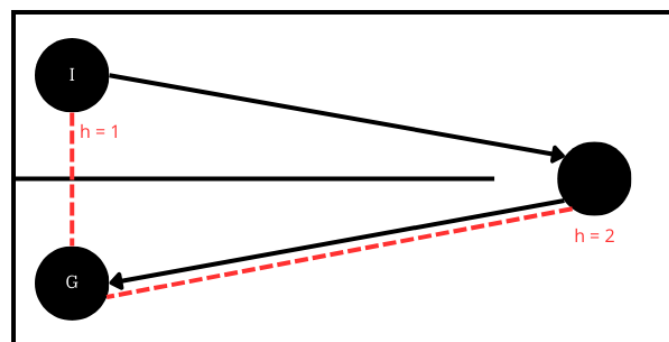


Figure 3: Euclidean heuristic failing in a labyrinth-like map.

1.6 Exercise 3.6

Inflating the heuristic means replacing $h(x)$ with $c \cdot h(x)$ where $c > 1$.

Consequences.

- For $c = 1$: A* is optimal.
- For $c \rightarrow 0$: the search becomes similar to Dijkstra, very slow but optimal.
- For $c \rightarrow \infty$: the search becomes greedy like Best First, very fast but suboptimal.

Plots. Figure 4 shows how plan length and planning time vary with inflation factor.

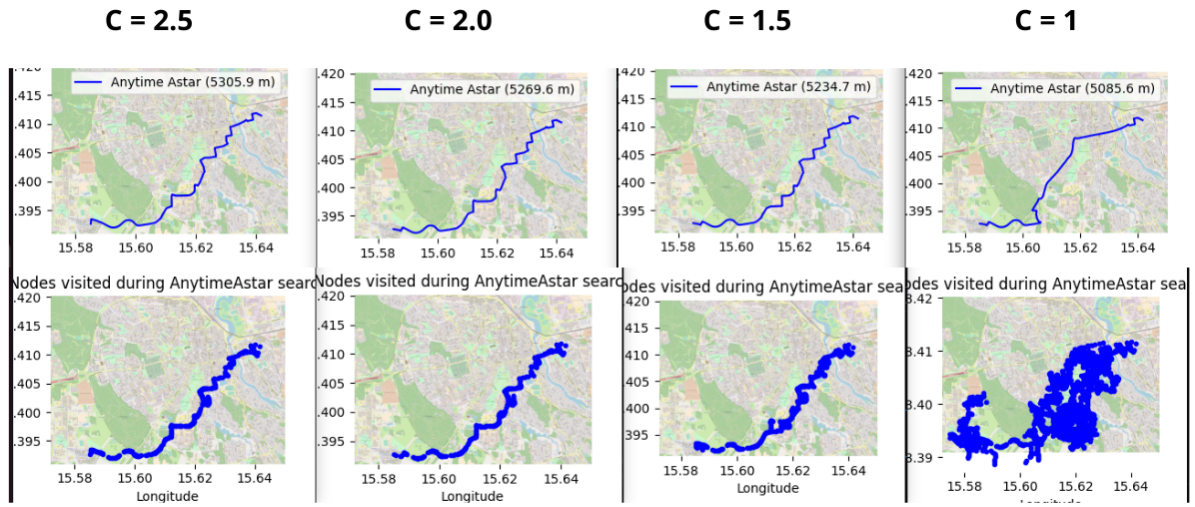


Figure 4: Influence of inflation factor on plan length and nodes expanded.

Observation. The experiments confirm that inflation allows controlling the trade-off: small c values yield longer runtimes with optimal solutions, while large c values reduce runtime but increase path cost.

2 APPENDIX

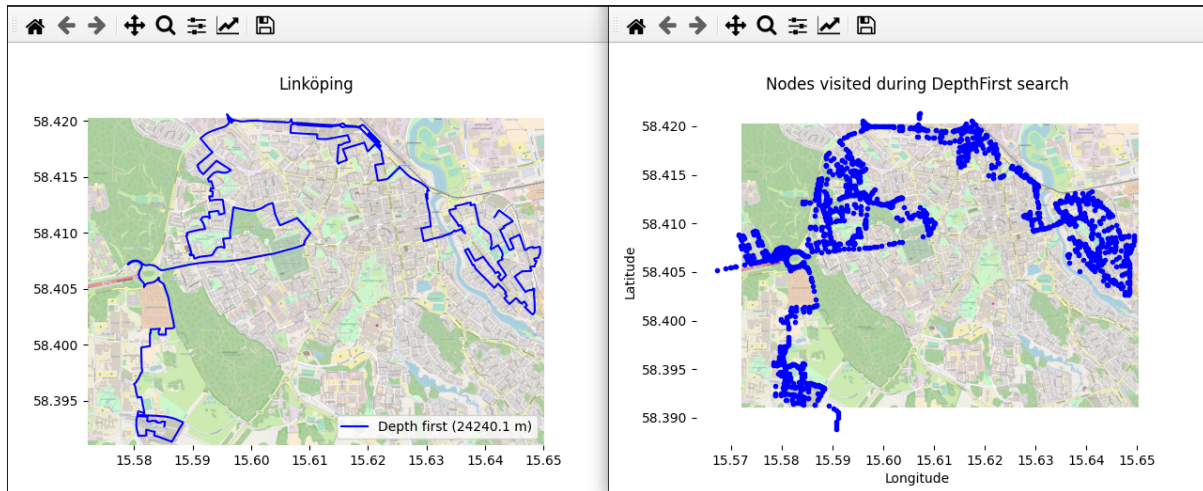


Figure 5: DepthFirst

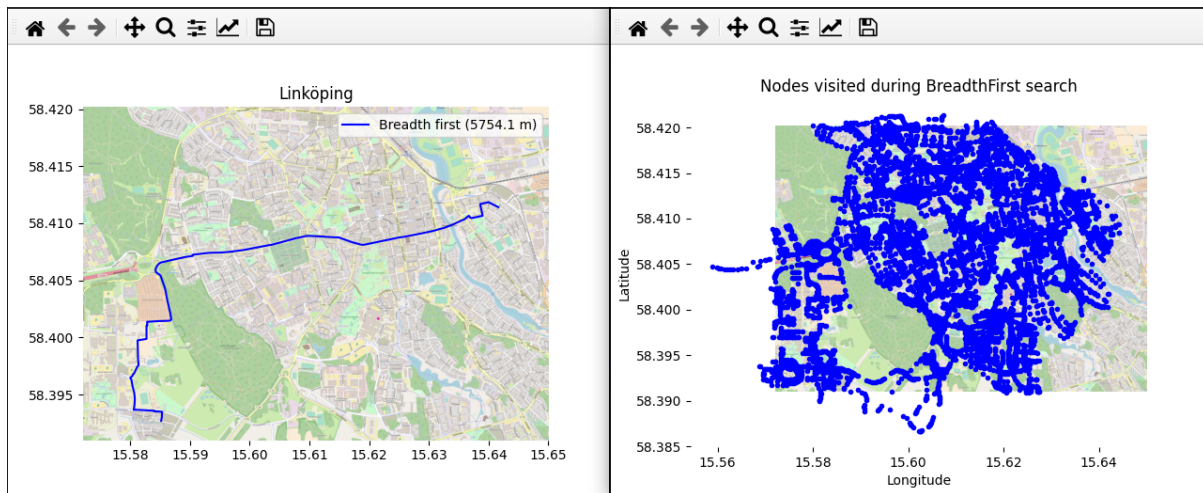


Figure 6: BreadthFirst

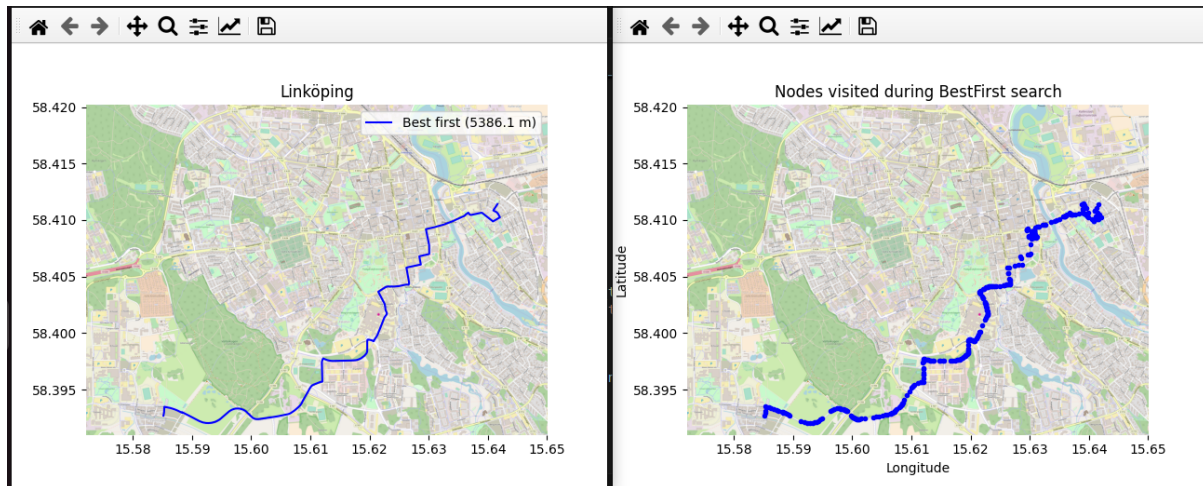


Figure 7: BestFirstResearch

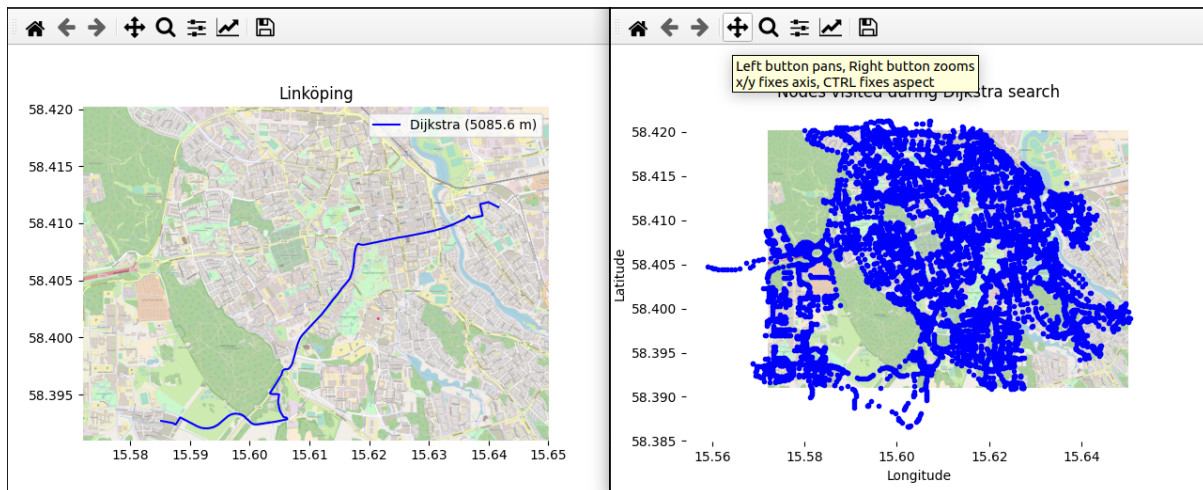


Figure 8: Dijkstra

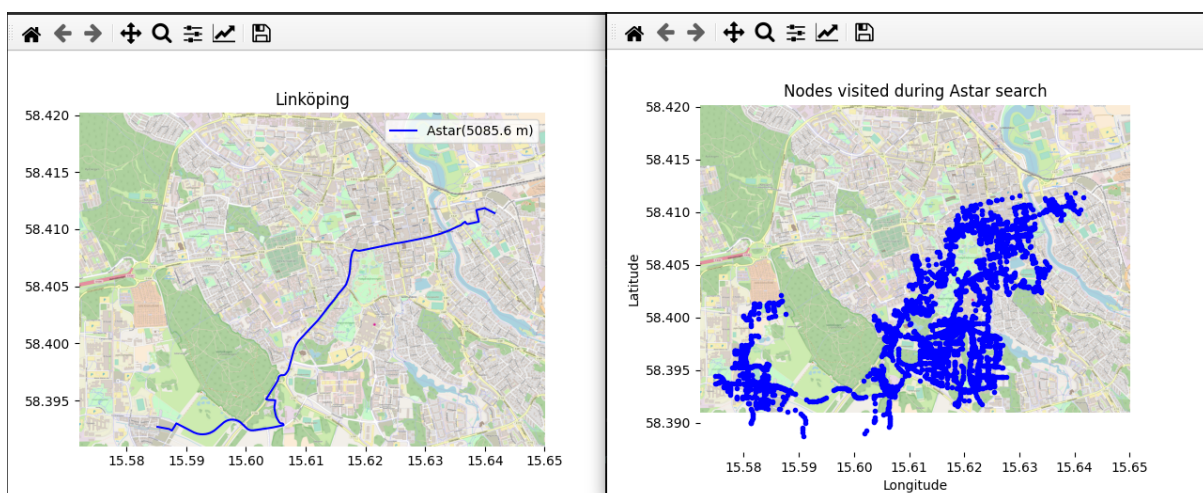


Figure 9: A*